

R.A.V.E.N — Functionalities

Everything R.A.V.E.N can do today, checked against the code on 2026-09-21 (tool roster, slash commands, settings and confirmation rules were read from the source, not recalled). For *why* things are the way they are, see `DEV_LOG.md` ; for setup, see `README.md` .

R.A.V.E.N is a terminal assistant that talks to an LLM through OpenRouter and lets it act on your machine through a fixed set of tools. The model decides which tool to call; anything consequential goes through a confirmation prompt first.

Contents: [Interface](#) • [Agent tools](#) • [Confirmation model](#) • [Memory and context](#) • [Model and configuration](#) • [Prerequisites](#) • [Known limitations](#) • [Verification notes](#) • [Where things live](#)

Interface

Slash commands

Typed at the `>` prompt. They run directly, without the LLM. Tab-completion suggests them as you type `/` .

Command	What it does	Confirms?
<code>/help</code>	Show the command table	—
<code>/status</code>	Show CPU, memory and disk usage	—
<code>/read</code> <code>&lt;path></code>	Print a file's contents	no
<code>/write</code> <code>&lt;path></code>	Write content to a file (overwrites)	no

Command	What it does	Confirms?
<content>		
/mkdir <path>	Create a directory	no
/run <command>	Run a shell command	always
/statusline [segments]	With no arguments, list segments and the current order; with names, set and save them	—
/forget	Clear conversation history, in memory and on disk	no
/compact	Summarize older history into a running summary now, freeing up context budget	no
/notes	Show saved cross-session notes (~/.raven/memory/notes.md)	—
/exit , /quit	Leave (plain exit / quit also work)	—

Anything that isn't a slash command goes to the model.

Headless mode

`raven -p "<prompt>"` runs one request non-interactively and exits — no TTY, no Rich styling, plain text on stdout, so it's pipeable and scriptable (cron, CI, shell scripts). Confirmation-gated tool calls are **denied by default** in this mode — headless never silently approves something consequential just because no one's watching — add `-y / --yes` to auto-approve them instead. History, settings, and `RAVEN.md` rules are shared with interactive mode; a real error prints and exits with status 1.

Keys

Key	Effect
<code>Ctrl+C</code>	Cancel the current request. The half-finished turn is rolled back so it can't confuse the next one.
<code>Ctrl+O</code>	Show or hide the model's reasoning trace for the last reply (collapsed by default). It lives in the bottom toolbar, so it toggles instantly with no new reply needed.
<code>Ctrl+V</code>	Toggle push-to-talk voice input — press to start recording, press again to stop, transcribe locally, and insert the text into the input buffer for review (never auto-sent). Needs optional packages; see "Voice input" below. Silently does nothing useful (inserts a "not set up" note) if they're missing.

While it works and after it replies

- **Live activity line.** A custom animated spinner alongside status text that shows what the agent is doing right now ("Reading x.txt...", "Searching the web for ...", "Opening figma.com in Firefox..."), changing as it moves between tool calls. While waiting on the model with no tool running, the text instead rotates through a set of idle words every 5s, each rendered with a brightness "shimmer" sweeping across it. The moment a tool call starts, that rotation pauses — a tool's own status sticks until either another tool call replaces it or the turn ends, rather than being overwritten by the next random word up to 5s later (a real bug, fixed 2026-09-23).
- **"Thought for X.Xs".** Printed after each successful reply, with a hint to press `Ctrl+O` when the model supplied a reasoning trace. Skipped when the request was cancelled or errored.
- **Reasoning trace.** Shown in the bottom toolbar when expanded, capped at 1200 characters.
- **Markdown rendering** of replies.
- **Startup.** A banner, a randomly chosen greeting, a "Resumed N messages from earlier sessions" line if history was loaded, and a "Loaded rules from ..." line if a `RAVEN.md` was found (see "Project/user rules" below).

Bottom status line

Always visible under the prompt. Segments, in the order set by `/statusline` (default is the first four):

Segment	Shows
<code>model</code>	The model that <i>actually answered</i> the last request (which can differ from the one requested, if a fallback took over)
<code>context</code>	History messages sent to the model / the cap (<code>ctx 12/40</code>)
<code>tokens</code>	Cumulative tokens used this session
<code>memory</code>	System memory in use
<code>cpu</code>	System CPU load

It also shows `[ctrl+o: thoughts collapsed|expanded]` and refreshes every 2 s.

New segments are one function in `raven/statusline.py`.

Agent tools

The model has 35 tools (34 registered + `load_skill` itself), but only the ones in "Files and shell" below (plus `load_skill`) are sent on every request — everything under "Applications and windows", "Web", "Browser interaction", "Vision", and "Gmail" is grouped into a named **skill** (`desktop` , `web` , `browser` , `vision` , `gmail` respectively) that stays locked until the model calls `load_skill` for it, then works exactly like any other tool for the rest of that session (see "Skills" below for why). "Confirms" means you are asked to allow it before it runs; the prompt is built from what is about to happen.

Files and shell

Tool	What it does	Confirms?
<code>read_file</code>	Read a text file	no

Tool	What it does	Confirms?
<code>write_file</code>	Write a text file	only if it already exists (overwriting)
<code>edit_file</code>	Replace an exact, unique string in an existing file — a targeted change without rewriting the whole thing	yes, always (it only ever touches an existing file)
<code>make_dir</code>	Create a directory, parents included	no
<code>list_dir</code>	List a directory (<code>d</code> = dir, <code>-</code> = file)	no
<code>tree</code>	Recursive directory structure, indented by depth (bounded to 3 levels / 500 entries)	no
<code>glob_files</code>	Find files by name/glob pattern, e.g. <code>**/*.py</code> (for content, use <code>grep</code>)	no
<code>grep</code>	Search text files by regex (<code>file:line:</code> results)	no
<code>save_note</code>	Save one fact to durable, cross-session memory (see "Memory and context")	no
<code>delete_file</code>	Delete a single file (not directories)	yes
<code>run_command</code>	Run a shell command, 30 s timeout	yes, unless it is a plain call to a read-only program (below)
<code>run_tests</code>	Run a test command (default <code>pytest</code>), 120 s timeout	same policy as <code>run_command</code>

Tool	What it does	Confirms?
<code>git</code>	Run a git subcommand in the current repo	yes, unless read-only (<code>status</code> , <code>log</code> , <code>diff</code> , <code>show</code>)

`edit_file`'s uniqueness rule: `old_string` must match exactly once in the file — zero matches or more than one both refuse outright rather than guess, so a vague target (e.g. a common one-line string that appears twice) never risks editing the wrong spot. The model is expected to include enough surrounding context to pin down a single location, the same rule Claude Code's own edit tool follows.

Read-only commands that skip confirmation: `ls` , `pwd` , `whoami` , `date` , `uname` , `df` , `du` , `ps` , `wc` , `which` , `cat` , `head` , `tail` , `echo` , `id` , `hostname` , `free` , `uptime` . This only applies when the command contains **no shell metacharacters** (`;` | `&` < `>` `$()` or a newline) — so `ls; rm -rf ~`` and `echo hi > file` always ask. The same guard applies to `git` , so `git log ; rm -rf ~` asks too.

User-configurable permissions (C3) — `~/.raven/settings.json`'s `permissions` section extends this trust additively, never in place of it:

```
{"permissions": {"allow_commands": ["pytest"], "allow_git": ["commit", "add"]}}
```

`allow_commands` lets a plain (still metacharacter-free) call to that program — via `run_command` or `run_tests` — skip confirmation, on top of the built-in read-only list above. `allow_git` does the same for git subcommands, on top of `status` / `log` / `diff` / `show` . The shell-metacharacter check always still applies regardless of this list: `"pytest; rm -rf ~"` still confirms even with `pytest` trusted — trusting a program name only ever exempts a plain call to exactly that program.

Hooks (D1) — the same `settings.json` can run a shell command automatically before and/or after a specific tool call:

```
{"hooks": {"post": {"edit_file": "black {path}"}, "pre": {"edit_file":
```

```
"cp {path} {path}.bak"}}
```

{placeholder} s are filled from that tool call's own arguments. Never confirmed — being listed in this local, user-controlled file is the approval, same trust boundary as permissions . A post hook is skipped if the tool call's own result started with "Error:" (nothing actually changed, so nothing to act on); a hook's output or error is appended to the tool's own result rather than replacing or hiding it. Scoped to the model's own tool-calling loop only — /write , /run , and the other manual slash commands bypass this entirely, same as they already bypass some of the Assistant-level confirmation logic.

Applications and windows

Tool	What it does	Confirms?
list_apps	List every installed app's display name	no
open_app	Launch an installed app by name	no
open_file	Open a file with the system default app, or with a named app	no
close_app	Close a running app — every window of it (process signal)	yes
list_windows	List open windows (id, app, title)	no
close_window	Close exactly one window, matched by title	yes

How app handling works:

- **Discovery** reads .desktop files from native packages, Flatpak (system and per-user) and Snap, so apps from any of those are found.
- **Matching is fuzzy but strict.** Whole-word, both directions ("the text editor app" matches "Text Editor"). An ambiguous name returns the candidates instead of guessing; an unknown name returns the closest installed names.
- **VS Code opens files and folders in the window you already have (-r),** rather than spawning a new one each time. Other apps use their normal

launch.

- `close_window` is the only per-window control. It needs the [Window Calls](#) GNOME Shell extension; without it the tool says so instead of failing.

Web

Tool	What it does	Confirms?
<code>open_url</code>	Open a URL in the default browser, or a named one (e.g. Firefox)	no
<code>fetch_url</code>	Fetch a page and return its text (HTML stripped, ~4000 chars, 10 s timeout)	no
<code>web_search</code>	Search DuckDuckGo, return the top 5 titles, URLs and snippets	no

Browser interaction

A separate, persistent, **headless and isolated** Chromium (Playwright). It has none of your real browser's cookies or logins, so it can read and interact with public pages but can't act as you on sites you're signed into. It is started on first use and stays alive across calls, so `navigate` → `read` → `click` → `type` → `submit` acts on one continuous page. Elements are targeted by **visible text or label**, not CSS.

Tool	What it does	Confirms?
<code>browser_navigate</code>	Load a URL (JavaScript-rendered pages work, unlike <code>fetch_url</code>)	no
<code>browser_read</code>	Return the page's visible text (~4000 chars)	no

Tool	What it does	Confirms?
<code>browser_click</code>	Click a button or link	no — but it refuses a real form-submit control and redirects to <code>browser_submit</code>
<code>browser_type</code>	Type into a field by its label or placeholder	no
<code>browser_submit</code>	Click a control that commits an action (submit, buy, send, log in)	yes
<code>browser_close</code>	Close the browser session	no
<code>browser_screenshot</code>	Save a PNG of the current page to a temp file, return its path — pair with <code>analyze_image</code> to actually see it	no

Vision

Tool	What it does	Confirms?
<code>analyze_image</code>	Look at an image file (screenshot, photo, diagram) and answer a question about it	no

Uses a **separate**, configurable model (`settings.json` 's `model.vision`), not the main chat model — the default may not support images at all. A full-desktop screenshot (rather than just R.A.V.E.N's own browser page) isn't supported yet; it needs the Wayland/GNOME screenshot portal's more involved async permission flow.

Gmail

Real inbox access through the Gmail API (OAuth). Granted scopes are exactly `gmail.readonly` and `gmail.send`, so Google itself refuses to delete or alter existing mail.

Tool	What it does	Confirms?
<code>gmail_list_messages</code>	List recent messages (id, date, sender, subject); accepts Gmail search syntax such as <code>is:unread</code> or <code>from:x@y.com</code>	no
<code>gmail_read_message</code>	Read one message's sender, date, subject and body (~4000 chars)	no
<code>gmail_send_message</code>	Send a new email	yes — prompt shows the full To, Subject and Body
<code>gmail_reply_message</code>	Reply to an existing email, in its thread, to its <code>Reply-To</code> (else <code>From</code>); plain reply, not reply-all	yes — prompt shows the resolved recipient, Subject and Body

For a reply, the prompt shows where the mail will really go, not just the visible sender, because a sender can set `Reply-To` elsewhere. If that lookup fails, the prompt still appears. A recipient containing a line break is refused.

Voice input (V1)

`Ctrl+V` toggles push-to-talk: press to start recording, press again to stop. Transcribed locally and offline via [faster-whisper](#) — no network call, no OpenRouter quota used — and inserted into the input buffer for review/editing, never auto-sent.

Needs two optional Python packages (`sounddevice` , `faster-whisper`) plus, **on Linux only**, the system PortAudio library:

```
pip install sounddevice faster-whisper
sudo apt install libportaudio2 # Debian/Ubuntu; dnf/pacman equivalents
on other distros
```

This is a real, unavoidable requirement for ANY Linux user who wants voice input — `sounddevice` 's Linux wheel doesn't bundle PortAudio the way its Windows/macOS wheels do, so without the system library it fails at import with `OSError: PortAudio library not found` . None of it is required to run R.A.V.E.N at all: missing dependencies just make `Ctrl+V` insert a "voice input not set up" note instead of recording — everything else is completely unaffected. The model weights (~140MB) are downloaded once via the explicit `python -m raven.voice` command (use `HF_HUB_DISABLE_XET=1` if it stalls at 0 bytes) — a keypress never downloads anything; until then `Ctrl+V` shows "voice model not downloaded" in the toolbar. Transcription runs on a background thread so the UI never blocks. `Ctrl+C` at the prompt clears the line; `Ctrl+D` on an empty prompt exits. `Ctrl+V` is voice, not paste — use the terminal's paste shortcut (usually `Ctrl+Shift+V`).

Skills (D2)

Only "core" (file I/O, search, shell, coding, `save_note` — 13 tools) plus `load_skill` itself are sent to the model on every request; `desktop` , `web` , `browser` , and `gmail` (19 tools total) are grouped skills, loaded on demand. This exists to fix a measured, fixed cost: every registered tool's full spec (name, description, parameter schema) is normally sent on **every** request regardless of what that conversation is about — measured at 2.9k tokens for 26 tools back when this was first flagged (DEV_LOG, 2026-09-21), now ~4k for 32. Splitting that into core + on-demand skills, and moving each skill's guardrail prose (the Gmail Reply-To/untrusted-content rules, the browser submit-safety rule, the app-name-matching guidance) out of the always-sent system prompt into text appended only once that skill is actually loaded, measured at roughly **halving** the fixed overhead (5.6k → ~2.75k tokens) for a conversation that only ever touches core tools.

How it works: `load_skill(name)` is a special tool, handled by `Assistant` itself rather than `tools.py` — it doesn't touch the outside world, it changes what's offered to the model next. Calling it adds that skill's tool names to the next request's tool list, and (for a real provider) appends its guardrail text to the live `system_prompt` in place, so nothing else needs to change to pick it up on the following request. Never asks for confirmation (it's not consequential — it only expands what's available, exactly the way core tools already were), and is idempotent (loading an already-loaded skill just says so). Nothing about a skill's tools changes once loaded — same confirmation policy as always; this only affects **when** their specs and instructions reach the model, not how they behave. A skill's activation is per-session, in-memory only, same reasoning as B2/B3's decision not to persist compaction summaries or notes-loading state — a fresh session always starts back at core-only, and re-loading a skill if/when it's needed again is cheap and correct either way.

Confirmation model

Eleven tools can ask for permission. What each guard actually is:

Tool	Asks when	Enforced by
<code>write_file</code>	only if the target already exists (overwriting)	code
<code>edit_file</code>	always (only ever touches an existing file)	code
<code>delete_file</code>	always	code
<code>close_app</code>	always ("close ALL windows of X")	code
<code>close_window</code>	always	code
<code>run_command</code>	unless a read-only program or a pipeline of read-only programs (e.g. <code>pdftotext f.pdf - </code>	code (allow-list plus

Tool	Asks when	Enforced by
	head), or user-trusted (permissions.allow_commands)	metacharacter check)
run_tests	same policy as run_command	code
git	unless status / log / diff / show , or user-trusted (permissions.allow_git), with no metacharacters	code
browser_submit	always	code
gmail_send_message	always	code
gmail_reply_message	always	code

A denied request returns "Cancelled by user", and the model is instructed to say plainly that nothing was done. The model is also instructed **not** to ask in prose first, since the prompt already exists.

Where the boundary is model-dependent, not code-enforced:

- browser_click only catches *literal* HTML submit controls. A JavaScript "Buy Now" that commits without being a <button type=submit> would not be caught by it; the model is told to use browser_submit for anything that commits.
 - Choosing close_window over close_app , gmail_reply_message over gmail_send_message , or open_url with an app name over the default browser is the model's call, guided by tool descriptions.
 - **Prompt-injection defence is instruction-only.** The model is told that web page content and email content are data, never instructions. The confirmation prompts above are the hard backstop for anything consequential.
-

Memory and context

- **Persistent history** in `~/ .raven/history.db` (SQLite), loaded on startup. Only completed turns are saved; a cancelled or errored turn is not.
- **Bounded context.** Only as much recent history as fits an ~12k-token budget (a chars/4 approximation) is sent to the model, trimmed so the window never starts on an orphaned tool result. This is a practicality budget for a slow free tier, not a model context-window limit. Full history is kept in memory and on disk regardless.
- **Auto-compaction.** Whatever falls out of that budget isn't just dropped — it's summarized into a single running summary, sent in place of the raw turns it replaces. Compaction runs automatically whenever the budget boundary advances (cached and only extended incrementally, so a stable conversation doesn't re-summarize every turn), or on demand with `/compact`. It's a dedicated LLM call, capped at 2000 summary characters as a backstop. Crucially, it never touches the real, persisted history — `self.history` and `~/ .raven/history.db` stay the full raw record; only what's *sent per request* is affected, so a fresh session always starts from the untouched original and nothing is ever actually lost, only compressed for the model's sake.
- **Bounded tool output.** Genuinely unbounded tool results (`read_file` , `run_command` , `list_dir`) are capped at ~4000 characters, with a "truncated, N more chars" marker — the same convention `fetch_url` , `browser_read` , and Gmail message bodies already used.
- `/forget` wipes it. History can go stale (an app you installed after the model said it wasn't there), so the model is told to re-run the relevant tool instead of trusting an earlier conclusion, but `/forget` is the reset.
- **Cross-session notes** (`save_note` , `~/ .raven/memory/notes.md`) are a separate, durable memory the model writes to on its own initiative — a preference, a convention, a correction — outside conversation history entirely, so they survive `/forget` and are loaded into every future session's system prompt. Bounded on the write path rather than trusted to the model's judgment: one note capped at ~500 characters, the file at

~4000; both refuse outright rather than truncate a fact or evict an older note once full. Never needs confirmation (append-only, bounded, and the file is just plain markdown the user can read or edit by hand — see `/notes`).

- **Tool loop cap:** at most 10 tool-call rounds per request. Past that it stops and says so, rather than looping.
- **No emojis.** Stripped from replies and reasoning in code, since free models routinely ignore the instruction not to use them.

Project/user rules

An optional `RAVEN.md` in the current working directory, and/or an optional `~/ .raven/RAVEN.md` , are loaded into the system prompt once at startup — project-specific conventions (coding standards, how to run tests, anything the model should know about that project) or personal preferences that apply everywhere. Verified live: a rule in a project's `RAVEN.md` ("always end every reply with X") was genuinely followed by the model, not just appended to the prompt text.

- If both are present, user rules come first and project rules last, so the more specific one wins on anything they disagree on.
- Each file is capped at ~4000 characters (same convention as tool output).
- An empty, missing, or unreadable file is silently treated as absent — never an error.
- R.A.V.E.N reports on startup which files, if any, were loaded.

Model and configuration

- **Provider:** OpenRouter. Requests use a model list, so if the primary is down, rate-limited or paywalled, the next is tried automatically. Defaults: `nvidia/nemotron-3.5-lightning:free` , falling back to `google/gemma-4-31b-it:free` , then `openrouter/free` .
- **Requests time out after 60 s** and OpenRouter's error text is surfaced, not hidden behind a generic HTTP error.
- **Token accounting** comes from the API's `usage` figures.

- **Settings** (all optional) live in `~/.raven/settings.json`: `model.name`, `model.fallbacks`, `statusline.segments`.
- **API key:** `OPENROUTER_API_KEY`, from the environment or a `.env` file.
- **Swappable provider:** implement `LLMProvider.reply()` in `raven/llm_provider.py` and change one line in `cli.py`.
- **Persona:** professional, calm and concise. The system prompt also carries what the name stands for (Reasoning Agent for Versatile Execution & Negotiation).
- **Project/user rules:** an optional `RAVEN.md` in the working directory and/or `~/.raven/RAVEN.md` are loaded into the system prompt at startup — user rules first, project rules last (closer to the task, so they win on conflict). Each capped at ~4000 characters. Reported on startup when found.

Files R.A.V.E.N reads or writes

Path	Purpose
<code>.env</code> (project)	<code>OPENROUTER_API_KEY</code>
<code>RAVEN.md</code> (project)	Optional project-specific rules, loaded into the system prompt
<code>~/.raven/RAVEN.md</code>	Optional user-wide rules, loaded into every project's system prompt
<code>~/.raven/history.db</code>	Conversation history
<code>~/.raven/settings.json</code>	Model and status-line settings (optional)
<code>~/.raven/gmail_credentials.json</code>	Your Google OAuth client secret (you provide it)
<code>~/.raven/gmail_token.json</code>	Cached Gmail login, created on first consent

Prerequisites

Not everything works out of the box; some tools depend on your system.

Needed for	Requires
Everything	Python 3.10+, an OpenRouter API key
App launching and file opening	<code>gtk-launch</code> , <code>xdg-open</code> (Linux desktop)
<code>close_app</code>	<code>pkill</code>
<code>list_windows</code> / <code>close_window</code>	<code>gdbus</code> and the Window Calls GNOME Shell extension (GNOME)
Browser tools	<code>playwright</code> and a one-time <code>playwright install chromium</code> (~115 MB)
Gmail tools	A Google Cloud project with the Gmail API, an OAuth Desktop client, and your account added as a test user — a one-time setup only you can do (see README, "Gmail access")

Known limitations

Missing safeguards

- `/run` always asks, even for commands the agent itself runs without asking (`ls` and the like). The two paths use different rules.

Platform

- Built for a **Linux desktop**. App handling relies on `.desktop` files, `gtk-launch` and `xdg-open` ; window control is **GNOME + Wayland + the Window Calls extension** specifically. It will not work as-is on macOS or Windows.
- `close_app` closes **all** windows of an app. Per-window closing is only `close_window` .

Web and browser

- The isolated browser **cannot see your real browser** — not your open tabs, not your logged-in sessions. `open_url` can open a page in your real Brave or Firefox, but only as fire-and-forget; nothing can be read back from it.
- `web_search` uses DuckDuckGo's free, keyless endpoint, which rate-limits after a burst of searches; it then returns a "temporarily rate-limited" message.

Gmail

- No reply-all, forward, drafts, attachments, or HTML composing (plain-text bodies only). Cannot delete or modify existing mail.
- Reading prefers plain text, falling back to HTML with tags stripped; attachments are ignored.

Model

- Runs on free OpenRouter models: latency varies, the model that answers can change from call to call, and there is a daily request cap.

Not built yet

- Voice *output* (text-to-speech) — voice *input* is built (see "Voice input" above).
 - A full-desktop screenshot for vision — only R.A.V.E.N's own browser page can be captured (`browser_screenshot`) until the Wayland/GNOME screenshot portal's async flow is built.
 - `analyze_image` 's live model call is unverified as of 2026-09-23 — built and unit-tested, but never yet confirmed against a real vision model (blocked by OpenRouter's daily quota that day); check DEV_LOG before assuming it works end to end.
 - Streaming replies (text appearing as it is generated).
 - Searching or recalling *past* sessions — history is one continuous conversation, not searchable.
 - Task-specific modes (coding / research / planning) — Stage 4 on the roadmap.
 - An HTTP/API wrapper for other front-ends — Stage 5.
-

Verification notes

Nearly everything above was exercised live during development (see `DEV_LOG.md`, which records each check). The exceptions worth knowing:

- **Sending mail has never been run for real.** `gmail_send_message` and `gmail_reply_message` were verified up to the API's send call — message construction, threading headers, recipient resolution, and the confirmation preview against a fake Gmail service, plus the read and metadata calls against the real API — but no real email has been sent, since that would message a real person as you. Try one to yourself first.
- **Gmail body reading** (`gmail_read_message`) was verified against fabricated message payloads, not against real message bodies.
- **VS Code window reuse** was verified at the argument level (the exact command is `code -r <path>`); a real VS Code window was deliberately not launched to confirm the reuse.
- **Ctrl+O live toggle** was verified at the function level (same data, different output) but not observed repainting in a real terminal from the development side.

Where things live

File	Responsibility
<code>raven/cli.py</code>	Entry point, slash commands, key bindings, live activity line, rendering
<code>raven/assistant.py</code>	Conversation state, the tool-call loop, the confirmation policy, emoji stripping, context trimming
<code>raven/llm_provider.py</code>	The system prompt, <code>RAVEN.md</code> rules loading, the provider interface, the OpenRouter client

File	Responsibility
<code>raven/tools.py</code>	File, shell, search, app, window and web tools, plus the tool registry and specs
<code>raven/browser.py</code>	The Playwright browser session (runs on its own thread)
<code>raven/gmail.py</code>	Gmail OAuth, listing, reading, sending and replying
<code>raven/store.py</code>	SQLite history persistence
<code>raven/statusline.py</code>	Bottom status-line segments
<code>raven/config.py</code>	Environment and <code>settings.json</code> loading
<code>tests/</code>	The pytest suite — one file per module, plus <code>conftest.py</code> for shared fixtures